

Cambios en el método tipoOperacion



En este video existe un cambio en el código del método tipoOperación, ya no es posible simplemente indicar el tipo evento como parámetro del método tipoOperación, ahora es necesario indicar el tipo de dato, además de hacer una conversión explícita del tipo de dato esperado y con el tipo de dato con el que se trabaja. Aquí la explicación de por qué hacer este cambio:

El cambio que experimentaste en las últimas versiones de Angular con respecto al manejo de tipos en TypeScript está relacionado con las mejoras y ajustes que Angular y TypeScript han implementado para promover mejores prácticas y aumentar la robustez del código.

Código anterior:

```
tipoOperacion(evento){  
    this.tipo = evento.target.value;  
}
```

Nuevo código:

```
tipoOperacion(evento: Event){  
    const target = evento.target as HTMLSelectElement;  
    this.tipo = target.value;  
}
```

Cambios en las últimas versiones de Angular y TypeScript:

1. Aumento de las configuraciones estrictas de TypeScript:

- En versiones más recientes, Angular ha adoptado configuraciones más estrictas de TypeScript por defecto. Esto incluye la configuración de strict y noImplicitAny, que requieren que todos los tipos sean explícitos.
- Esta configuración es beneficiosa porque ayuda a prevenir errores al exigir que todos los parámetros y variables tengan tipos bien definidos, lo que mejora la seguridad y claridad del código.

2. Mejoras en el análisis estático:

- Las versiones más nuevas de TypeScript tienen mejores capacidades de análisis estático, lo que permite detectar más posibles problemas en el código. Esto incluye situaciones donde no se especifica un tipo y el compilador asume any, lo que puede llevar a errores no detectados en tiempo de desarrollo.

3. Buenas prácticas promovidas:

- Al requerir que los tipos sean explícitos, Angular y TypeScript fomentan mejores prácticas de desarrollo. Por ejemplo, en tu caso, asegurar que evento es de tipo Event y que evento.target se convierte explícitamente en HTMLSelectElement previene errores potenciales si se trabaja con diferentes tipos de eventos o elementos en el DOM.

Por qué funcionaba antes:

En versiones anteriores de Angular y TypeScript, las configuraciones por defecto eran menos estrictas. La configuración noImplicitAny podía estar desactivada o simplemente TypeScript permitía que los parámetros sin tipo explícito se asumieran como any sin generar advertencias o errores. Esto daba más flexibilidad, pero también podía introducir errores que eran difíciles de detectar.

¿Por qué el cambio?

El objetivo de estos cambios es aumentar la calidad del código. Al exigir que los tipos sean explícitos, se reduce la posibilidad de errores, especialmente en aplicaciones grandes y complejas. Esto es parte de una tendencia más amplia en el ecosistema de JavaScript y TypeScript hacia la tipificación estricta y el desarrollo más seguro.

En resumen, el cambio que experimentaste es parte de la evolución hacia un desarrollo más seguro y robusto, promovido tanto por Angular como por TypeScript, a medida que estas herramientas continúan madurando.

Ing. Ubaldo Acosta

Universidad Angular

Saludos!

Ing. Ubaldo Acosta

Fundador de GlobalMentoring.com.mx